

# End-to-End Data Analytics Platform

Microsoft Fabric | Public Data | Spain Indicators

Complete technical specification defining environment setup, DDL scripts, notebook structures, Dataflow Gen2 configurations, pipeline orchestration, authentication, logging framework, DAX measures, and deployment strategy.

| AUTHOR             | STACK                 | DATE                 |
|--------------------|-----------------------|----------------------|
| Ssr Data Analyst   | Microsoft Fabric      | April 2026           |
| 6 years experience | Lakehouse   Warehouse | Duration: 1-2 months |
| Microsoft Stack    | PySpark   Power BI    | Version 1.0          |

# 1. Executive Summary

This document defines the technical specification for the End-to-End Data Analytics Platform. It is the implementation blueprint that translates the functional specification (Phase 3) into concrete engineering instructions: how the environment is provisioned, how objects are created, how notebooks are structured, how pipelines are orchestrated, how credentials are handled, and how the solution is deployed from development to production.

Where Phase 3 defined **WHAT** the system must produce (business rules, derived measures, source-to-target field mappings), this document defines **HOW** those outputs are engineered inside Microsoft Fabric. The two documents are complementary and must be read together: Phase 3 owns business intent; Phase 4 owns technical implementation.

## 1.1 Document Scope

This specification covers the technical design at the level of a professional engineering blueprint. It deliberately stops short of producing final production code. Code in its final form is authored during Phase 5 (Development), where it is iterated against the live Fabric environment and versioned in Git during Phase 6 (Publication).

| In scope for this document                                                         | Out of scope (deferred to Phase 5 / 6)                                                  |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Environment provisioning (Azure, Fabric workspaces, Lakehouse, Warehouse)          | Production-ready PySpark source code (authored in Phase 5 against the live environment) |
| Complete DDL scripts for the Warehouse star schema                                 | Final Power Query M code for Dataflows Gen2 (exported from Fabric in Phase 6)           |
| Notebook structural specification (sections, functions, libraries, error handling) | Pipeline JSON/YAML export files (generated by Fabric, versioned in Phase 6)             |
| Dataflow Gen2 step-by-step logical design (Power Query steps in prose)             | Screenshots of the live Fabric workspace (captured during Phase 5)                      |
| Pipeline orchestration design (activities, parameters, dependencies, retry)        | Final TMDL-format export files from the semantic model (generated in Phase 5)           |
| Authentication architecture (cross-tenant Fabric to Azure SQL)                     | End-user documentation for the Power BI report                                          |
| Logging framework implementation (DDL + helper function signatures)                | Performance tuning results (measured after Phase 5)                                     |
| Complete DAX measure definitions and formulas                                      | Incremental loading patterns (explicitly out of project scope)                          |
| Deployment strategy (DEV to PROD via Fabric Deployment Pipelines)                  | CI/CD automation (explicitly out of project scope)                                      |
| Validation queries and testing checklist per layer                                 | Monitoring and alerting (explicitly out of project scope)                               |

## 1.2 Why This Level of Detail

A technical specification written at the level of final code would be simultaneously premature and wasteful. **Premature**, because implementation inevitably reveals constraints that only emerge in the live environment: connector behavior, Direct Lake limitations, Dataflow Gen2 UI constraints, and Fabric-specific quirks that cannot be anticipated from documentation alone. **Wasteful**, because Phase 6 (GitHub Publication) versions the final code in detail with commit history, inline comments, and per-component README files — duplicating that effort here would produce two sources of truth that drift over time.

The appropriate analogy is an architectural blueprint for a building: the specification defines load-bearing walls, electrical paths, materials, and finishes, but does not prescribe the exact sequence in which the bricks are laid. This document is the blueprint. Phase 5 is the construction. Phase 6 is the as-built documentation.

***Guiding principle** — every technical decision in this document is stable enough to survive implementation unchanged in the majority of cases. Where implementation reveals a better path, the deviation is documented as a 'Phase 5 implementation decision' in the GitHub repository, preserving the original rationale for traceability.*

### 1.3 Document Structure

The document follows the natural sequence an engineer would traverse when implementing the solution from scratch: environment setup first, then standards and conventions, then objects (Warehouse DDL), then components (notebooks, dataflows, pipelines), then operational concerns (connections, logging), then consumption (Power BI), then operationalization (deployment, testing, troubleshooting).

| #  | Section                          | Focus                                                 |
|----|----------------------------------|-------------------------------------------------------|
| 1  | Executive Summary                | Scope, methodology, and document structure            |
| 2  | Setup and Prerequisites          | Azure and Fabric environment provisioning             |
| 3  | Code Conventions and Standards   | Notebook structure, naming, styling, logging pattern  |
| 4  | Gold Layer DDL                   | Complete T-SQL for fact and dimension tables          |
| 5  | Notebooks Specification          | Structural design per notebook (Bronze, Silver, Gold) |
| 6  | Dataflows Gen2 Specification     | Power Query step design for IPC and IPI               |
| 7  | Data Pipelines Orchestration     | Master, Bronze, Silver, Gold pipeline design          |
| 8  | Connections and Authentication   | Cross-tenant Fabric to Azure SQL configuration        |
| 9  | Logging Framework Implementation | Log table DDL and helper function pattern             |
| 10 | Power BI Technical Specification | Direct Lake setup, relationships, DAX catalog         |
| 11 | Deployment Strategy              | DEV to PROD via Fabric Deployment Pipelines           |
| 12 | Testing and Validation Plan      | Layer-by-layer verification queries and checklist     |
| 13 | Troubleshooting Guide            | Common errors and resolution patterns                 |
| 14 | Next Steps                       | Transition to Phase 5 Development                     |

### 1.4 Audience

This document is written for a technical audience: data engineers, analytics engineers, and solution architects responsible for building or reviewing the solution. Business audiences should refer to Phase 3 (Functional Specification). Project governance and scope audiences should refer to Phase 0 (Project Brief).

## 1.5 Relationship to Other Phases

| Upstream dependency                       | What this document inherits                                                                        |
|-------------------------------------------|----------------------------------------------------------------------------------------------------|
| Phase 0 — Project Brief                   | Scope boundaries, quality standards, capacity licensing strategy                                   |
| Phase 1 — Requirements and Source Catalog | Five source definitions, access patterns, technical challenges per source                          |
| Phase 2 — Architecture Design             | Medallion architecture, component mapping, object naming conventions, layered orchestration design |
| Phase 3 — Functional Specification        | Source-to-target mappings, star schema fields, derived measure rules, data quality rules           |
| Downstream consumer                       | What this document delivers                                                                        |
| Phase 5 — Development                     | Complete implementation blueprint: DDL, notebook structure, pipeline activities, DAX catalog       |
| Phase 6 — Publication                     | Source-of-truth for repository organization and component-level README structure                   |

## 2. Setup and Prerequisites

This section defines everything that must exist before the first line of code is written in Phase 5. It covers two independent environments that must be provisioned and connected: the Azure subscription hosting the SQL Database (source system for IPI), and the Microsoft Fabric workspace hosting the Lakehouse, Warehouse, and all compute components.

**Setup sequencing** — Azure provisioning is performed **before** activating the Fabric trial. The Fabric trial is time-bounded and should be activated only when development is ready to begin. Every hour of the trial consumed on setup activities that could have been done beforehand is an hour lost to actual development.

### 2.1 Environment Overview

The platform uses two distinct cloud environments that belong to different Microsoft Entra tenants. This is not accidental: it reflects a realistic consulting scenario in which the analytics workspace (Fabric) and an upstream transactional system (Azure SQL) are governed by different organizations.

| Environment                | Tenant                    | Identity                         | Purpose                                                                      |
|----------------------------|---------------------------|----------------------------------|------------------------------------------------------------------------------|
| Azure subscription         | Personal (author-owned)   | Personal Microsoft account       | Hosts the Azure SQL Database that stores the IPI staging table               |
| Microsoft Fabric workspace | Employer corporate tenant | Employer-issued Entra ID account | Hosts the Lakehouse, Warehouse, notebooks, pipelines, semantic model, report |

■ ■ **Cross-tenant architecture** — because the two environments live in separate tenants, the Fabric-to-Azure-SQL connection cannot use Entra ID single sign-on. The connection is configured using **SQL authentication** (username and password stored in the SQL Database). Section 8 specifies this configuration in detail and documents it as a deliberate portfolio highlight rather than a limitation.

### 2.2 Azure Subscription and SQL Database

#### 2.2.1 Azure Account Creation

A personal Azure subscription is created using a personal Microsoft account. The employer corporate email is explicitly not used, for three reasons: (a) the resulting subscription would be bound to the employer tenant and potentially governed by its IT policies, (b) access would be lost on employment termination, (c) mixing personal portfolio resources with corporate identity creates ambiguity around ownership.

**STEP 1 — Register an Azure account.** Navigate to <https://azure.microsoft.com/free> and sign up using a personal Microsoft account. A credit card is required for identity verification but no charges are incurred while usage stays within the free allowances documented in section 2.2.2.

**STEP 2 — Confirm subscription creation.** Once registration completes, a subscription of type 'Free Trial' (for the first 30 days) and/or 'Pay-As-You-Go' is available. Either is compatible with the Azure SQL Database free offer. The Azure portal opens at <https://portal.azure.com>.

## 2.2.2 Azure SQL Database — Free Offer

The IPI staging database is provisioned under the Azure SQL Database free offer, which provides a serverless General Purpose database at no cost within its monthly allowance. The allowance is more than sufficient for portfolio usage: the IPI dataset is approximately 40,000 rows loaded once, and Fabric reads from it on demand during pipeline execution.

| Free offer allowance       | Value                           | Project usage estimate                       |
|----------------------------|---------------------------------|----------------------------------------------|
| Compute                    | 100,000 vCore-seconds per month | Well under 5,000 vCore-seconds per month     |
| Data storage               | 32 GB per database              | Well under 50 MB                             |
| Backup storage             | 32 GB per database              | Not actively used                            |
| Databases per subscription | Up to 10                        | One database required                        |
| Service tier               | General Purpose serverless      | Fits the portfolio use case                  |
| Maximum compute            | 4 vCores, auto-pause enabled    | Compatible with occasional reads from Fabric |
| Duration                   | Lifetime of the subscription    | No expiration risk                           |

***Auto-pause billing protection** — at database creation, the option 'Auto-pause the database until next month' is selected. If monthly allowances are ever exhausted, the database pauses until the calendar resets and no charges are incurred. The alternative option ('Continue usage with additional charges') is explicitly not selected.*

## 2.2.3 Database Provisioning

**STEP 3 — Create a resource group.** Region: West Europe (lowest latency from Barcelona and compatible with most Fabric region pairings). Name: rg-spi-portfolio.

**STEP 4 — Create a logical SQL Server.** Name: sql-spi-ipi-source. Authentication: 'Use both SQL and Microsoft Entra authentication'. SQL server admin login and password recorded in a personal password manager. Entra admin set to the author's personal Microsoft account.

**STEP 5 — Create the SQL Database (free offer).** Server: the logical server from Step 4. Database name: sqldb\_ipi\_source. Workload environment: Development. Compute tier: Serverless. Free offer: **enabled**. Free limit behavior: **Auto-pause the database until next month**. Collation: SQL\_Latin1\_General\_CP1\_CI\_AS.

**STEP 6 — Configure firewall rules.** In the SQL Server networking blade: add the author's client IP as an allowed rule, and enable 'Allow Azure services and resources to access this server'. The latter is required for Fabric (Microsoft-managed cloud service) to reach the database. Fabric-specific outbound IP ranges are not used for trial capacity scenarios.

## 2.2.4 IPI Data Load Into Azure SQL

The IPI source is originally an INE CSV (Source 2 in the functional spec). To simulate a transactional source system, the CSV is loaded once into a staging table in sqldb\_ipi\_source. Fabric then reads from this staging table as if it were an enterprise transactional database.

| Step             | Tool            | Action                                                                                  |
|------------------|-----------------|-----------------------------------------------------------------------------------------|
| Download         | Browser or curl | Fetch the IPI CSV from the INE URL defined in the Phase 1 source catalog. Save locally. |
| Convert encoding | Python or iconv | Convert the CSV from ISO-8859-15 to UTF-8 to ensure clean ingestion.                    |

| Step                 | Tool                      | Action                                                                                                          |
|----------------------|---------------------------|-----------------------------------------------------------------------------------------------------------------|
| Create staging table | Azure Data Studio or SSMS | DDL defined below creates the staging table with all columns typed as NVARCHAR to mirror the raw CSV structure. |
| Bulk insert          | Azure Data Studio or bcp  | Load the UTF-8 CSV into the staging table using the appropriate field separator (semicolon) and row terminator. |
| Validate             | T-SQL                     | Row count check, distinct values on comunidades_y_ciudades_autonomas, sample rows inspection.                   |

### Staging table DDL:

```
CREATE TABLE stg_ipi_raw (
    comunidades_y_ciudades_autonomas NVARCHAR(200) NOT NULL,
    destino_economico_de_los_bienes NVARCHAR(200) NOT NULL,
    tipo_de_dato NVARCHAR(100) NOT NULL,
    periodo NVARCHAR(10) NOT NULL,
    total NVARCHAR(50) NULL,
    _loaded_at DATETIME2(0) NOT NULL
    CONSTRAINT DF_stg_ipi_raw_loaded DEFAULT SYSUTCDATETIME()
);
GO

CREATE INDEX IX_stg_ipi_raw_ccaa
ON stg_ipi_raw (comunidades_y_ciudades_autonomas);
GO
```

*Why NVARCHAR for all columns — the staging table mirrors the CSV exactly: every column is stored as a string. This protects against unexpected source format changes (stray non-numeric values in total, locale-specific decimal separators). Type casting happens in the Fabric Silver layer where it can be logged and validated.*

## 2.3 Microsoft Fabric Environment

### 2.3.1 Capacity and License

Fabric development uses the corporate Fabric Trial Capacity approved by the author's employer. The trial grants access to all Fabric workloads with up to 64 Capacity Units and 1 TB of OneLake storage. The trial is bounded in time; Section 2.5 details the time-management strategy.

| Attribute          | Value                                                                                        |
|--------------------|----------------------------------------------------------------------------------------------|
| Capacity type      | Fabric Trial (F64 equivalent)                                                                |
| Duration           | 60 days standard (extendable by 60 days when less than 7 days remain)                        |
| Workloads included | All (Data Engineering, Data Factory, Data Warehouse, Power BI, Real-Time Analytics)          |
| OneLake storage    | 1 TB                                                                                         |
| Identity           | Employer-issued Entra ID account (author's corporate identity)                               |
| Governance         | Author retains full ownership of all artifacts; nothing confidential to the employer is used |

### 2.3.2 Workspace Topology

Two workspaces are provisioned following the naming convention from the Phase 2 architecture document. Each workspace is independently assigned to the trial capacity.

| Workspace | Name                      | Role                                               | Data scope                                  |
|-----------|---------------------------|----------------------------------------------------|---------------------------------------------|
| DEV       | spi-spain-indicators-dev  | Active development and iteration                   | Last 24 periods per source (fast iteration) |
| PROD      | spi-spain-indicators-prod | Final portfolio deployment and screenshot evidence | Full historical dataset per source          |

### 2.3.3 Core Fabric Items

The following items are created inside each workspace. The DEV workspace is provisioned first; PROD is provisioned near the end of Phase 5 via a Fabric Deployment Pipeline (specified in Section 11).

| Item type                       | Name                           | Purpose                                                     |
|---------------------------------|--------------------------------|-------------------------------------------------------------|
| Lakehouse                       | spi_lakehouse                  | Stores Bronze and Silver Delta tables and the logging table |
| Warehouse                       | spi_warehouse                  | Hosts the Gold star schema (1 fact + 5 dimensions)          |
| Semantic Model                  | spi_spain_indicators           | Direct Lake semantic model built on spi_warehouse           |
| Power BI Report                 | spi_spain_indicators           | Six-page report per Phase 3 specification                   |
| Notebooks, Pipelines, Dataflows | Per Phase 2 naming conventions | Created during Phase 5 as implementation progresses         |

### 2.3.4 Why Warehouse for Gold, Not Lakehouse SQL Endpoint

The Gold layer is implemented in Fabric Warehouse rather than as Delta tables in the Lakehouse (consumed via its SQL analytics endpoint). The decision is deliberate and reflects three considerations:

- **Portfolio breadth.** Using both Lakehouse and Warehouse demonstrates familiarity with both Fabric compute engines and their respective use cases. A portfolio that uses only the Lakehouse leaves the Warehouse track of the DP-600 certification unevidenced.
- **Star schema appropriateness.** The Warehouse supports T-SQL DDL with primary keys, foreign keys, and constraints (declared but non-enforced) that are natural expressions of a dimensional model. The Lakehouse SQL endpoint does not expose the same DDL surface.
- **Enterprise pattern alignment.** The standard enterprise pattern of 'Lakehouse for raw and cleaned data, Warehouse for dimensional model' mirrors the Bronze-Silver in Lakehouse and Gold in Warehouse split used here. This is the pattern most common in production Fabric deployments and the one most clients will ask for.

*Alternative considered — an all-Lakehouse architecture (Gold as Delta tables, consumed via SQL endpoint in Direct Lake) is a valid modern pattern and would simplify the layer transition by removing the Lakehouse-to-Warehouse hop. It is not chosen here because it does not exercise the Warehouse surface area and dilutes the portfolio demonstration. This note preserves the rationale for future reference.*

## 2.4 Development Tooling (Local)

Several activities during Phase 5 benefit from local tools in addition to the Fabric web interface. These are listed for completeness and are not strictly required for the Fabric-native workflow.

| Tool                      | Purpose                                                         | Required / Recommended   |
|---------------------------|-----------------------------------------------------------------|--------------------------|
| Visual Studio Code        | Editing PBIP files and Markdown documentation                   | Recommended              |
| Git (CLI or GUI)          | Version control of exported artifacts and docs                  | Required (Phase 6)       |
| Azure Data Studio or SSMS | Managing the Azure SQL Database, running the IPI bulk load      | Required (Section 2.2.4) |
| Power BI Desktop          | Optional local editing of the PBIP report prior to publishing   | Recommended              |
| Python 3.x (local)        | Optional: convert the IPI CSV encoding locally before uploading | Optional                 |



## 2.5 Trial Time Management Strategy

Because Phase 5 development must complete within the Fabric trial window, provisioning and preparatory steps that can be done outside the trial are pushed out of the development window. This maximizes the available engineering hours.

| Activity                                                 | Trial active? | When                                 |
|----------------------------------------------------------|---------------|--------------------------------------|
| Create Azure subscription                                | No            | Before Phase 5 kickoff               |
| Provision Azure SQL Database and load IPI CSV            | No            | Before Phase 5 kickoff               |
| Draft all Phase 4 content (DDL, notebook structure, DAX) | No            | This document (Phase 4)              |
| Activate Fabric trial                                    | Day 1         | Phase 5 kickoff                      |
| Create DEV workspace, Lakehouse, Warehouse               | Yes           | Phase 5 Day 1-2                      |
| Implement and test five source pipelines                 | Yes           | Phase 5 Week 1-5                     |
| Deploy DEV to PROD, capture screenshots                  | Yes           | Phase 5 final week                   |
| Publish artifacts to GitHub                              | Optional      | Phase 6 (artifacts already exported) |

**Target timeline** — the development window targets **4 to 6 weeks of active Phase 5 work**, aiming at mid-June 2026 for a completed portfolio artifact published to GitHub. This sits comfortably inside the trial window without rushing.

## 3. Code Conventions and Standards

Consistency across all Fabric components is essential for a portfolio project. A reviewer should be able to open any notebook, pipeline, or dataflow and immediately recognize the same structure, naming, and logging pattern. This section defines those conventions as cross-cutting rules applied throughout Phase 5.

### 3.1 Naming Conventions

Naming inherits directly from Phase 2 (Architecture Design, section 8) and is repeated here for self-contained reference.

| Object type             | Pattern                            | Example                            |
|-------------------------|------------------------------------|------------------------------------|
| Global prefix           | spi_                               | spi_lakehouse                      |
| Tables and columns      | snake_case, lowercase              | indicator_value, region_name       |
| Bronze tables           | spi_bronze__raw                    | spi_bronze_ipc_raw                 |
| Silver tables           | spi_silver__cleaned                | spi_silver_ipc_cleaned             |
| Gold fact               | spi_fact_                          | spi_fact_indicators                |
| Gold dimensions         | spi_dim_                           | spi_dim_region                     |
| Notebooks               | spi_nb__                           | spi_nb_bronze_energy               |
| Pipelines               | spi_pl_                            | spi_pl_master, spi_pl_bronze       |
| Dataflows Gen2          | spi_df__                           | spi_df_silver_ipc                  |
| Log table               | Operational prefix                 | spi_log_pipeline_execution         |
| Bronze metadata columns | Underscore prefix                  | _ingestion_timestamp, _source_file |
| Workspaces              | kebab-case with environment suffix | spi-spain-indicators-dev           |

### 3.2 Notebook Structure

Every notebook in the project follows the same five-section structure. This uniformity makes reviewing easier and keeps new notebooks consistent with the existing ones.

| Section       | Purpose                                                                                                                                     |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Header     | Metadata markdown cell with notebook name, layer, source, owner, purpose, and last-updated date                                             |
| 2. Imports    | All library imports grouped: standard library first, third-party second, Fabric/PySpark utilities last                                      |
| 3. Parameters | Parameterized cell (tagged 'parameters' for pipeline injection). Defines inputs such as workspace, lakehouse name, date ranges, source URLs |
| 4. Functions  | All reusable logic defined as pure functions with docstrings. No side effects inside function definitions                                   |
| 5. Main       | Orchestration block that calls functions in order, writes output, and updates the logging table. Wrapped in try/except to capture errors    |

**Rationale** — separating functions from main enables unit-style testing of individual transformations without running the full ingestion. It also makes the main block short and readable: a reviewer can understand the notebook's flow in under a minute.

### 3.3 PySpark Style

| Rule                                                                           | Rationale                                            |
|--------------------------------------------------------------------------------|------------------------------------------------------|
| Use snake_case for all Python variables, function names, and column names      | Consistency with the rest of the project and PEP 8   |
| Prefer DataFrame API over SQL strings for transformations                      | Easier to read, compose, and test in isolation       |
| Explicit schema definitions when reading semi-structured data (JSON, XLSX)     | Avoids silent schema drift when source changes       |
| Short, single-purpose functions (20-30 lines maximum)                          | Testability and readability                          |
| Docstrings on every function (1-2 lines describing input, output, side effect) | Enables notebook-to-doc extraction in Phase 6        |
| No magic numbers: constants defined at the top of the notebook                 | Makes parameters visible and changeable in one place |
| Type hints on function signatures when reasonably possible                     | Documents intent and helps IDE tooling               |

### 3.4 Logging Pattern

Every notebook and every pipeline must write to the centralized log table `spi_log_pipeline_execution` (specified in Section 9). The pattern is uniform across components:

- **Start:** insert a row with `status = 'running'` and `start_time = now`.
- **Work:** execute the notebook's main logic.
- **End (success):** update the row with `status = 'success'`, `end_time = now`, `rows_processed = N`.
- **End (failure):** update the row with `status = 'failed'`, `end_time = now`, `error_message = <exception>`.

A shared helper module (specified in Section 9) exposes three functions: `log_start()`, `log_end_success()`, `log_end_failure()`. Every notebook imports and uses these helpers rather than writing raw logging logic.

### 3.5 Error Handling Philosophy

| Principle                                                  | Application                                                                                                                 |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Catch and log errors at the notebook boundary              | Each notebook wraps its main block in try/except, logs the failure, and re-raises the exception so the pipeline captures it |
| Do not swallow errors silently inside functions            | Functions either succeed or raise; no partial-state fallbacks                                                               |
| Data quality violations produce warnings, not exceptions   | Rows failing DQ rules are rejected (or preserved with warning per Phase 3 section 6.1); the notebook continues              |
| Transient errors (network, API timeouts) use bounded retry | Only where documented (REE API); file/DB errors do not retry                                                                |
| Error messages must be actionable                          | Include the source, the step, and what was expected vs. received                                                            |

### 3.6 Comments and Documentation

The balance is: **documentation where code cannot speak for itself.**

- Write comments for *why*, not *what*. The code shows what; comments explain decisions, tradeoffs, or source quirks.
- Use markdown cells in notebooks to separate major stages and explain business rules or source-specific gotchas.

- Every notebook has a top markdown cell summarizing purpose and dependencies. Every pipeline has a description field filled in.
- Dead code is deleted, not commented out. Git preserves history.

## 4. Gold Layer DDL

This section defines the complete T-SQL DDL for the six Warehouse tables that implement the star schema. The DDL is stable and can be executed as-is in Phase 5.

**Note on Fabric Warehouse constraints** — Fabric Warehouse supports primary key and foreign key declarations, but they are **NOT ENFORCED** (informational only). They are declared here because (a) they document the intended model for reviewers, (b) they enable the Power BI semantic model to detect relationships automatically, (c) they align with industry-standard data modeling practices.

### 4.1 Schema Organization

All Gold objects live in the dbo schema of spi\_warehouse. A dedicated schema (e.g. gold) is not used because Fabric Warehouse schema management adds complexity without benefit at this scale.

### 4.2 Fact Table — spi\_fact\_indicators

```
CREATE TABLE dbo.spi_fact_indicators (  
    indicator_key    INT            NOT NULL,  
    region_key       INT            NOT NULL,  
    calendar_key     INT            NOT NULL,  
    measure_key      INT            NOT NULL,  
    source_key       INT            NOT NULL,  
    indicator_value  DECIMAL(18,4) NULL,  
    CONSTRAINT PK_fact_indicators PRIMARY KEY NONCLUSTERED (  
        indicator_key, region_key, calendar_key, measure_key, source_key  
    ) NOT ENFORCED,  
    CONSTRAINT FK_fact_indicator  
        FOREIGN KEY (indicator_key)  
        REFERENCES dbo.spi_dim_indicator (indicator_key) NOT ENFORCED,  
    CONSTRAINT FK_fact_region  
        FOREIGN KEY (region_key)  
        REFERENCES dbo.spi_dim_region (region_key) NOT ENFORCED,  
    CONSTRAINT FK_fact_calendar  
        FOREIGN KEY (calendar_key)  
        REFERENCES dbo.spi_dim_calendar (calendar_key) NOT ENFORCED,  
    CONSTRAINT FK_fact_measure  
        FOREIGN KEY (measure_key)  
        REFERENCES dbo.spi_dim_measure (measure_key) NOT ENFORCED,  
    CONSTRAINT FK_fact_source  
        FOREIGN KEY (source_key)  
        REFERENCES dbo.spi_dim_source (source_key) NOT ENFORCED  
);
```

| Column          | Type               | Notes                                                        |
|-----------------|--------------------|--------------------------------------------------------------|
| indicator_key   | INT NOT NULL       | FK to spi_dim_indicator                                      |
| region_key      | INT NOT NULL       | FK to spi_dim_region                                         |
| calendar_key    | INT NOT NULL       | FK to spi_dim_calendar, format YYYYMM                        |
| measure_key     | INT NOT NULL       | FK to spi_dim_measure (IDX, YoY, MoM, YTD, QTD)              |
| source_key      | INT NOT NULL       | FK to spi_dim_source                                         |
| indicator_value | DECIMAL(18,4) NULL | NULL allowed for derived measures per Phase 3 business rules |

### 4.3 Dimension — spi\_dim\_indicator

```
CREATE TABLE dbo.spi_dim_indicator (
    indicator_key      INT          NOT NULL,
    indicator_name     NVARCHAR(200) NOT NULL,
    indicator_category NVARCHAR(200) NOT NULL,
    domain             NVARCHAR(100) NOT NULL,
    unit_of_measure    NVARCHAR(100) NOT NULL,
    frequency          NVARCHAR(50)  NOT NULL,
    CONSTRAINT PK_dim_indicator PRIMARY KEY NONCLUSTERED (indicator_key) NOT ENFORCED
);
```

### 4.4 Dimension — spi\_dim\_region

```
CREATE TABLE dbo.spi_dim_region (
    region_key      INT          NOT NULL,
    region_code     NVARCHAR(3)  NOT NULL,
    region_name     NVARCHAR(50) NOT NULL,
    region_type     NVARCHAR(50) NOT NULL,
    CONSTRAINT PK_dim_region PRIMARY KEY NONCLUSTERED (region_key) NOT ENFORCED
);
```

Seed data (three rows, inserted once in Phase 5):

| region_key | region_code | region_name | region_type          |
|------------|-------------|-------------|----------------------|
| 1          | NAC         | Nacional    | Aggregated National  |
| 2          | CAT         | Cataluña    | Autonomous Community |
| 3          | MAD         | Madrid      | Autonomous Community |

### 4.5 Dimension — spi\_dim\_calendar

```
CREATE TABLE dbo.spi_dim_calendar (
    calendar_key      INT          NOT NULL,
    year              INT          NOT NULL,
    month             INT          NOT NULL,
    month_name        NVARCHAR(20) NOT NULL,
    quarter           INT          NOT NULL,
    year_month        NVARCHAR(7)  NOT NULL,
    is_current_year    BIT          NOT NULL,
    CONSTRAINT PK_dim_calendar PRIMARY KEY NONCLUSTERED (calendar_key) NOT ENFORCED
);
```

calendar\_key is a smart surrogate: the integer YYYYMM. This enables direct arithmetic filtering in T-SQL and DAX without joins. The dimension is programmatically populated covering 2000 through the current year (approximately 324 rows).

*Note on schema alignment — this dimension does not contain a date column. The grain is monthly, so individual days are not meaningful. Phase 3 section 4.4 already defines this model; Phase 4 implements it verbatim.*

### 4.6 Dimension — spi\_dim\_measure

```
CREATE TABLE dbo.spi_dim_measure (
    measure_key      INT          NOT NULL,
    measure_code     NVARCHAR(10) NOT NULL,
    measure_name     NVARCHAR(100) NOT NULL,
    measure_description NVARCHAR(500) NOT NULL,
    CONSTRAINT PK_dim_measure PRIMARY KEY NONCLUSTERED (measure_key) NOT ENFORCED
);
```

Seed data (five rows, inserted once in Phase 5):

| key | code | name                       | description                                                                    |
|-----|------|----------------------------|--------------------------------------------------------------------------------|
| 1   | IDX  | Index / Original Value     | Original value as provided by the source                                       |
| 2   | YoY  | Year-over-Year Rate        | % change vs. same month of previous year                                       |
| 3   | MoM  | Month-over-Month Rate      | % change vs. immediately preceding month                                       |
| 4   | YTD  | Year-to-Date Cumulative    | Cumulative % change from Jan to current month vs. same period of previous year |
| 5   | QTD  | Quarter-to-Date Cumulative | Cumulative % change from quarter start vs. same period of previous year        |

#### 4.7 Dimension — spi\_dim\_source

```
CREATE TABLE dbo.spi_dim_source (
    source_key          INT          NOT NULL,
    source_code         NVARCHAR(20) NOT NULL,
    source_name         NVARCHAR(200) NOT NULL,
    source_institution  NVARCHAR(100) NOT NULL,
    source_domain       NVARCHAR(100) NOT NULL,
    source_url          NVARCHAR(500) NULL,
    update_frequency    NVARCHAR(50) NOT NULL,
    CONSTRAINT PK_dim_source PRIMARY KEY NONCLUSTERED (source_key) NOT ENFORCED
);
```

Seed data (five rows, inserted once in Phase 5):

| key | code         | name                        | institution  | domain                |
|-----|--------------|-----------------------------|--------------|-----------------------|
| 1   | IPC          | Consumer Price Index        | INE          | Consumer Prices       |
| 2   | IPI          | Industrial Production Index | INE          | Industrial Production |
| 3   | ENERGY       | Electricity Demand          | REE (Redeia) | Energy                |
| 4   | CONSTRUCTION | Public Construction Tenders | MITMA        | Construction          |
| 5   | TAX          | Tax Revenue by Delegations  | AEAT         | Tax Revenue           |

#### 4.8 Execution Order

The DDL must be executed in dependency order: dimensions first (they have no FKs), then the fact table (which references all dimensions). The seed inserts for fixed dimensions (region, measure, source, calendar) run immediately after each dimension is created. The indicator dimension is populated during Phase 5 by the Gold notebook as sources are processed.

## 5. Notebooks Specification

This section defines the structural design of each notebook without prescribing final code. Every notebook follows the conventions of Section 3 (header, imports, parameters, functions, main) and uses the logging helpers of Section 9.

### 5.1 Notebook Inventory

| Notebook                   | Layer  | Source                 | Complexity  |
|----------------------------|--------|------------------------|-------------|
| spi_nb_bronze_energy       | Bronze | REST API (REE)         | Medium-High |
| spi_nb_bronze_construction | Bronze | XLS Legacy (MITMA)     | High        |
| spi_nb_bronze_tax          | Bronze | XLSX Complex (AEAT)    | High        |
| spi_nb_silver_energy       | Silver | Bronze Energy          | Medium      |
| spi_nb_silver_construction | Silver | Bronze Construction    | High        |
| spi_nb_silver_tax          | Silver | Bronze Tax             | High        |
| spi_nb_gold_load           | Gold   | All five Silver tables | High        |

*IPC and IPI do not have notebooks: they are handled end-to-end by Dataflows Gen2 (Section 6).*

### 5.2 Bronze Notebook — Energy (REE API)

**Purpose.** Ingest Spain's electricity demand evolution from the REE public REST API into `spi_bronze_energy_raw` (Delta table in Lakehouse).

#### Inputs.

| Parameter    | Description                                                                  |
|--------------|------------------------------------------------------------------------------|
| start_year   | First year to fetch (default: 2014)                                          |
| end_year     | Last year to fetch (default: current year)                                   |
| geo_entities | List of {geo_limit, geo_ids, geo_name} tuples for Nacional, Cataluña, Madrid |
| api_base_url | REE API base URL                                                             |

**Output.** Delta table `spi_bronze_energy_raw` with all fields as string + metadata columns (`_ingestion_timestamp`, `_source_file`).

**Key libraries.** `requests` (HTTP), `json` (parsing), `pyspark.sql` (DataFrame construction and Delta write), `time` (backoff sleep).

#### Functions (high-level inventory).

| Function                         | Responsibility                                                                                                        |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>build_request_url()</code> | Construct the API URL given year, geo_limit, geo_ids                                                                  |
| <code>fetch_with_retry()</code>  | Execute HTTP GET with 30s timeout, 3 retries with exponential backoff (2s, 4s, 8s), retry only on 429/500/502/503/504 |
| <code>parse_response()</code>    | Navigate the JSONAPI structure included[] → attributes.values[] and return a list of rows                             |
| <code>build_dataframe()</code>   | Convert the list of rows into a PySpark DataFrame with explicit schema                                                |
| <code>write_bronze()</code>      | Append to Delta table with schema merge disabled (schema stays fixed)                                                 |



**Error handling.** Transient HTTP errors trigger the retry policy. Permanent errors (400, 401, 404) fail immediately with a descriptive message. JSON parse failures include the offending response snippet in the error message. All errors propagate to the main block, which logs failure to `spi_log_pipeline_execution` and re-raises.

**Volume notes.** Three API calls per year (one per geo entity), approximately 12 years of data → ~36 calls per execution. Total rows in Bronze: ~500 per entity-year × 3 entities × 12 years = ~18,000 rows.

### 5.3 Bronze Notebook — Construction (XLS Legacy)

**Purpose.** Download four legacy XLS files from the MITMA statistical bulletin and consolidate them into `spi_bronze_construction_raw`.

#### Inputs.

| Parameter                          | Description                                                              |
|------------------------------------|--------------------------------------------------------------------------|
| <code>file_urls</code>             | Dict of {file_name: url} for the four XLS files                          |
| <code>contracting_agent_map</code> | Mapping from file name to contracting agent (State / Territorial)        |
| <code>region_map</code>            | Mapping from file name to geographic scope (Nacional+Cataluña vs Madrid) |

**Output.** Delta table `spi_bronze_construction_raw` with string fields + metadata.

**Key libraries.** `requests` (download), `xlrd` (read legacy .xls), `pandas` (intermediate parsing of merged cells), `pyspark.sql` (Delta write).

#### Functions (high-level inventory).

| Function                       | Responsibility                                                                                 |
|--------------------------------|------------------------------------------------------------------------------------------------|
| <code>download_file()</code>   | Fetch XLS from URL, save to temporary storage                                                  |
| <code>parse_xls_sheet()</code> | Open with <code>xlrd</code> , handle merged cells, extract multi-row header, return DataFrame  |
| <code>derive_metadata()</code> | Add <code>contracting_agent</code> and <code>region_name</code> columns from file-name mapping |
| <code>consolidate()</code>     | Concatenate the four DataFrames into one                                                       |
| <code>write_bronze()</code>    | Write consolidated DataFrame to Delta                                                          |

**Complexity notes.** Merged cells are detected via `xlrd`'s `merged_cells` attribute and expanded by forward-filling values. Multi-row headers are flattened into a single header row by concatenation. Subtotal rows (identified by label patterns like 'TOTAL', 'SUBTOTAL') are left in Bronze — they are removed in Silver.

**Error handling.** Per-file try/except; if one file fails, log the specific file and continue with the others. If zero files succeed, the notebook fails.

### 5.4 Bronze Notebook — Tax Revenue (XLSX Complex)

**Purpose.** Extract the `datos_delegaciones` sheet from the AEAT multi-sheet workbook and write it to `spi_bronze_tax_raw`.

#### Inputs.

| Parameter                 | Description                                 |
|---------------------------|---------------------------------------------|
| <code>workbook_url</code> | URL of the AEAT XLSX                        |
| <code>target_sheet</code> | Sheet name: <code>datos_delegaciones</code> |

**Output.** Delta table `spi_bronze_tax_raw` preserving the crosstab structure.

**Key libraries.** requests (download), openpyxl (read XLSX), pandas (sheet extraction), pyspark.sql (Delta write).

### Functions (high-level inventory).

| Function               | Responsibility                                                      |
|------------------------|---------------------------------------------------------------------|
| download_workbook()    | Fetch XLSX from URL                                                 |
| extract_target_sheet() | Open with openpyxl, read only datos_delegaciones, return raw values |
| to_dataframe()         | Build a DataFrame with header from row 1, all cells as strings      |
| write_bronze()         | Write DataFrame to Delta                                            |

**Complexity notes.** The workbook is ~6 MB but only one sheet is in scope. Opening with `read_only=True` and targeted sheet selection keeps memory usage low.

## 5.5 Silver Notebook — Energy

**Purpose.** Transform `spi_bronze_energy_raw` into `spi_silver_energy_cleaned`: flatten, deduplicate, type-enforce, filter per Phase 3 rules.

### Key transformations.

- Filter: `indicator_title = 'Demanda real'` only.
- Cast value to decimal; cast datetime → derive year, month.
- Map `geo_name` → `region_name` (Nacional, Cataluña, Madrid).
- Deduplicate by (`region_name`, `year`, `month`), keeping the latest by `_ingestion_timestamp`.
- Drop null / zero values per DQ-08 (with warning logged).

**Output columns.** `region_name`, `indicator_category` (constant 'Demanda real'), `year`, `month`, `indicator_value`.

## 5.6 Silver Notebook — Construction

**Purpose.** Transform `spi_bronze_construction_raw` into `spi_silver_construction_cleaned`.

### Key transformations.

- Remove subtotal rows (label-based filter).
- Normalize column names to `snake_case`.
- Parse `period_raw` → year, month.
- Cast `indicator_value_raw` → decimal.
- Preserve `contracting_agent` for downstream concatenation in Gold.
- Normalize `region_name` to the three canonical values.

## 5.7 Silver Notebook — Tax Revenue

**Purpose.** Transform `spi_bronze_tax_raw` (crosstab) into `spi_silver_tax_cleaned` (flat).

### Key transformations.

- **Unpivot:** geographic columns become rows. Retain only Total (Nacional), Madrid, Cataluña columns.
- Parse period info → year, month.

- Cast value to decimal.
- Map tax concept row labels to `indicator_category`.

## 5.8 Gold Notebook — Load

**Purpose.** Load all five Silver tables into the Gold star schema in `spi_warehouse`. Calculate the four derived measures (YoY, MoM, YTD, QTD) per Phase 3 business rules. Populate `spi_dim_indicator` as new indicators are encountered.

### Functions (high-level inventory).

| Function                             | Responsibility                                                                                                                                |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>read_silver_tables()</code>    | Load all five Silver tables into DataFrames                                                                                                   |
| <code>union_silver()</code>          | Concatenate into a unified DataFrame with a <code>source_code</code> column                                                                   |
| <code>upsert_indicator_dim()</code>  | For each unique ( <code>indicator_category</code> , <code>domain</code> ), insert into <code>spi_dim_indicator</code> if not present          |
| <code>lookup_dimension_keys()</code> | Join to each dimension to retrieve <code>indicator_key</code> , <code>region_key</code> , <code>calendar_key</code> , <code>source_key</code> |
| <code>compute_yoy()</code>           | Window function partitioned by ( <code>indicator_key</code> , <code>region_key</code> ), lag 12 months                                        |
| <code>compute_mom()</code>           | Window function partitioned by ( <code>indicator_key</code> , <code>region_key</code> ), lag 1 month                                          |
| <code>compute_ytd()</code>           | Cumulative sum from January of same year vs same period previous year                                                                         |
| <code>compute_qtd()</code>           | Cumulative sum from quarter start vs same period previous year                                                                                |
| <code>union_measures()</code>        | Concatenate IDX rows with derived measure rows, each with appropriate <code>measure_key</code>                                                |
| <code>write_warehouse()</code>       | Truncate and reload <code>spi_fact_indicators</code>                                                                                          |

**Data quality enforcement.** Rules DQ-01 through DQ-09 (Phase 3 section 6.1) are applied. Rejections are logged; warnings do not block load.

**Volume estimate.** ~60,000 Silver rows × 5 measures = ~300,000 fact rows after expansion. Well within Warehouse capacity.

## 6. Dataflows Gen2 Specification

Dataflows Gen2 handle the two simplest sources (IPC, IPI) where Spark overhead is not justified. This section describes the Power Query steps in prose order; final M code is produced in Phase 5 and exported in Phase 6.

### 6.1 Dataflow Inventory

| Dataflow               | Layer  | Source → Target                                                                                      |
|------------------------|--------|------------------------------------------------------------------------------------------------------|
| spi_df_silver_ipc      | Silver | spi_bronze_ipc_raw → spi_silver_ipc_cleaned                                                          |
| spi_df_silver_ipi      | Silver | spi_bronze_ipi_raw → spi_silver_ipi_cleaned                                                          |
| spi_df_gold_ipc        | Gold   | spi_silver_ipc_cleaned → rows in spi_fact_indicators                                                 |
| spi_df_gold_ipi        | Gold   | spi_silver_ipi_cleaned → rows in spi_fact_indicators                                                 |
| spi_df_gold_dimensions | Gold   | Seed inserts for spi_dim_region, spi_dim_measure, spi_dim_source, and generation of spi_dim_calendar |

### 6.2 Dataflow — spi\_df\_silver\_ipc (and IPI, analogous)

**Source.** Lakehouse table spi\_bronze\_ipc\_raw.

#### Steps (ordered).

1. **Connect** to the Bronze Delta table via Lakehouse connector.
2. **Filter rows** where tipo\_de\_dato = 'Indice' (discard pre-calculated variations).
3. **Filter rows** where comunidades\_y\_ciudades\_autonomas is in the list: 'Nacional', 'Cataluña', 'Madrid' (with normalization for accents and variants).
4. **Rename column** comunidades\_y\_ciudades\_autonomas → region\_name.
5. **Rename column** grupos\_ecoicop → indicator\_category.
6. **Split column** periodo (format YYYYMMNN) into year and month. Year = first 4 chars as integer. Month = substring after 'M' as integer.
7. **Rename column** total → indicator\_value.
8. **Change column type** of indicator\_value to decimal. Rows that fail the cast are rejected and logged.
9. **Remove columns:** drop Bronze metadata (\_ingestion\_timestamp, \_source\_file) — they are no longer needed in Silver.
10. **Destination:** write to Lakehouse as spi\_silver\_ipc\_cleaned, replace mode.

### 6.3 Dataflow — spi\_df\_gold\_ipc (and IPI, analogous)

**Source.** Lakehouse table spi\_silver\_ipc\_cleaned.

#### Steps (ordered).

1. **Connect** to the Silver Delta table.
2. **Lookup** indicator\_key from spi\_dim\_indicator (merge on indicator\_category and domain = 'Consumer Prices').
3. **Lookup** region\_key from spi\_dim\_region (merge on region\_name).

4. **Compute** `calendar_key` as `year * 100 + month (integer)`.
5. **Add constant columns:** `measure_key` = 1 (IDX), `source_key` = 1 (IPC for this dataflow; 2 for IPI).
6. **Select columns:** the five keys plus `indicator_value`.
7. **Destination:** append rows to Warehouse table `spi_fact_indicators`.

*Why IDX only here — YoY, MoM, YTD, QTD require window functions over the full fact table. They are computed by the Gold notebook (`spi_nb_gold_load`, Section 5.8) after all Silver tables have loaded their IDX rows. The dataflow only handles the IDX load for its specific source.*

## 6.4 Dataflow — `spi_df_gold_dimensions`

**Purpose.** Populate the fixed dimensions (`spi_dim_region`, `spi_dim_measure`, `spi_dim_source`) from static tables, and generate `spi_dim_calendar`.

**Approach.** Each dimension is loaded from a hardcoded table constructed in Power Query using the `Table.FromRecords()` pattern, with the seed data from Section 4. The calendar is generated by enumerating months from January 2000 to December of the current year.

**Execution.** Runs once at the start of Phase 5 development and does not need to re-run unless dimension definitions change.

## 7. Data Pipelines Orchestration

Pipelines implement the layered orchestration defined in Phase 2: a master pipeline invokes three layer pipelines (Bronze, Silver, Gold) which in turn invoke the appropriate notebooks and dataflows.

### 7.1 Pipeline Inventory

| Pipeline      | Role                                                            |
|---------------|-----------------------------------------------------------------|
| spi_pl_master | Top-level orchestrator; runs Bronze → Silver → Gold in sequence |
| spi_pl_bronze | Ingests all five sources into Bronze                            |
| spi_pl_silver | Transforms Bronze → Silver for all sources                      |
| spi_pl_gold   | Loads Silver → Gold Warehouse for all sources                   |

### 7.2 Pipeline — spi\_pl\_master

| Attribute  | Value                                                                     |
|------------|---------------------------------------------------------------------------|
| Activities | Three 'Invoke Pipeline' activities chained with 'On Success' dependencies |
| Activity 1 | Invoke spi_pl_bronze                                                      |
| Activity 2 | Invoke spi_pl_silver — runs only if Bronze succeeded                      |
| Activity 3 | Invoke spi_pl_gold — runs only if Silver succeeded                        |
| Parameters | environment (dev / prod), data_scope (last_24_periods / full_history)     |
| Logging    | Writes a single master log row at start and updates at end                |

### 7.3 Pipeline — spi\_pl\_bronze

| Activity | Type              | Runs                                            |
|----------|-------------------|-------------------------------------------------|
| 1        | Copy Activity     | IPC: INE URL (HTTP) → spi_bronze_ipc_raw        |
| 2        | Copy Activity     | IPI: Azure SQL stg_ipi_raw → spi_bronze_ipi_raw |
| 3        | Notebook Activity | Runs spi_nb_bronze_energy                       |
| 4        | Notebook Activity | Runs spi_nb_bronze_construction                 |
| 5        | Notebook Activity | Runs spi_nb_bronze_tax                          |

**Dependencies.** All five activities run in parallel (no inter-dependencies). Each is independent; failure of one does not block the others.

**On-error behavior.** 'Continue on error' at pipeline level. Each activity logs its own success/failure in spi\_log\_pipeline\_execution.

**Retry.** Only the Energy Notebook activity has pipeline-level retry enabled: 3 attempts with 30-second interval. This is redundant with the in-notebook retry (which handles HTTP-level failures); the pipeline retry handles catastrophic notebook failures (out-of-memory, kernel crash).

## 7.4 Pipeline — spi\_pl\_silver

| Activity | Type              | Runs                       |
|----------|-------------------|----------------------------|
| 1        | Dataflow Gen2     | spi_df_silver_ipc          |
| 2        | Dataflow Gen2     | spi_df_silver_ipi          |
| 3        | Notebook Activity | spi_nb_silver_energy       |
| 4        | Notebook Activity | spi_nb_silver_construction |
| 5        | Notebook Activity | spi_nb_silver_tax          |

**Dependencies.** Each Silver activity depends on its corresponding Bronze activity via the source table. In practice, because the master pipeline runs Bronze entirely before Silver, these activities run in parallel within the Silver pipeline.

**On-error behavior.** Continue on error within the layer.

## 7.5 Pipeline — spi\_pl\_gold

| Activity | Type              | Runs                                                                                                                |
|----------|-------------------|---------------------------------------------------------------------------------------------------------------------|
| 1        | Dataflow Gen2     | spi_df_gold_dimensions (populates fixed dimensions)                                                                 |
| 2        | Dataflow Gen2     | spi_df_gold_ipc (loads IPC IDX rows to fact)                                                                        |
| 3        | Dataflow Gen2     | spi_df_gold_ipi (loads IPI IDX rows to fact)                                                                        |
| 4        | Notebook Activity | spi_nb_gold_load (loads Energy, Construction, Tax IDX rows, and computes all derived measures for all five sources) |

**Dependencies.** Activity 1 must complete before any other Gold activity (dimensions are needed for FK lookups). Activities 2, 3, 4 run in parallel after Activity 1 succeeds. The notebook (Activity 4) is designed to handle its three sources plus the derived-measure computation for the IDX rows loaded by Activities 2 and 3; to ensure consistency, Activity 4 is configured to wait for Activities 2 and 3 to complete.

## 7.6 Pipeline Parameters

| Parameter   | Used by       | Values                                                                                 |
|-------------|---------------|----------------------------------------------------------------------------------------|
| environment | All pipelines | dev, prod                                                                              |
| data_scope  | All pipelines | last_24_periods, full_history                                                          |
| run_id      | All pipelines | GUID generated at master start, propagated to all child activities for log correlation |

## 7.7 Scheduling

No automatic schedule is configured. All pipelines are triggered manually during Phase 5 development. Production scheduling is explicitly out of scope per Phase 0 (Project Brief) and Phase 3 (Section 7 — Out of scope).

## 8. Connections and Authentication

This section specifies how Fabric connects to the Azure SQL Database, given that the two environments live in separate Microsoft Entra tenants.

### 8.1 Cross-Tenant Scenario

Fabric workspace identity (employer tenant) cannot authenticate via Entra ID against an Azure SQL Database in a personal tenant. Entra ID tokens are tenant-scoped. This is a constraint of the architecture, not a limitation of either product.

### 8.2 Authentication Method — SQL Authentication

The connection uses classic SQL authentication: a username and password stored inside the SQL Database. Fabric's Data Pipeline Copy Activity supports SQL authentication as a first-class connection type.

| Component | Configuration                                                                                        |
|-----------|------------------------------------------------------------------------------------------------------|
| Auth type | SQL Authentication                                                                                   |
| Username  | Dedicated read-only user: svc_fabric_reader                                                          |
| Password  | Strong password, stored in a Fabric-managed secret connection                                        |
| Network   | Public endpoint with firewall allow-list: 'Allow Azure services and resources to access this server' |

### 8.3 Dedicated Read-Only User

Rather than using the server admin credentials, a dedicated SQL user is created with read-only access to stg\_ipi\_raw only. This follows the principle of least privilege.

```
-- Executed once in the Azure SQL Database as admin
CREATE USER svc_fabric_reader WITH PASSWORD = '<strong-password>';
GRANT SELECT ON dbo.stg_ipi_raw TO svc_fabric_reader;
```

### 8.4 Fabric Connection Object

Inside Fabric, a **Connection** object of type 'SQL Server' is created once and reused by all pipeline activities that read from Azure SQL.

| Field           | Value                                   |
|-----------------|-----------------------------------------|
| Connection name | conn_azure_sql_ipi_source               |
| Server          | sql-spi-ipi-source.database.windows.net |
| Database        | sqldb_ipi_source                        |
| Authentication  | Basic (username + password)             |
| Credential      | svc_fabric_reader + stored password     |
| Encryption      | Required (TLS)                          |
| Privacy level   | Organizational                          |



## 8.5 Portfolio Value

The cross-tenant authentication scenario is called out in the GitHub README as a portfolio highlight. Real-world consulting engagements frequently involve analytics systems and operational data stores owned by different organizations. Documenting how the project handled this constraint is more valuable than hiding it behind a single-tenant demo setup.

## 9. Logging Framework Implementation

All notebook and pipeline executions write to a single central log table in the Lakehouse. This section specifies the table schema and the helper function pattern that notebooks and pipelines use to write to it.

### 9.1 Log Table DDL

The logging table lives in the Lakehouse (not the Warehouse) because it is operational metadata, not analytical data.

```
-- Created once via a Lakehouse notebook; executed as Spark SQL
CREATE TABLE spi_log_pipeline_execution (
  run_id          STRING,
  pipeline_name   STRING,
  layer           STRING,
  source          STRING,
  status          STRING,
  start_time      TIMESTAMP,
  end_time        TIMESTAMP,
  error_message   STRING,
  rows_processed  BIGINT
) USING DELTA;
```

| Column         | Meaning                                                       | Example                         |
|----------------|---------------------------------------------------------------|---------------------------------|
| run_id         | GUID generated by master pipeline, propagated to all children | a3f2...                         |
| pipeline_name  | Pipeline or notebook identifier                               | spi_nb_bronze_energy            |
| layer          | Medallion layer                                               | bronze                          |
| source         | Source identifier or 'all'                                    | energy                          |
| status         | Current state                                                 | running, success, failed        |
| start_time     | UTC timestamp of start                                        | 2026-04-10 14:30:00             |
| end_time       | UTC timestamp of completion (null while running)              | 2026-04-10 14:32:15             |
| error_message  | Exception or DQ warning summary                               | DQ-08: 3 negative values in IPC |
| rows_processed | Rows written by this execution                                | 45230                           |

### 9.2 Helper Function Pattern

A shared notebook (or %run-included module) exposes three functions used by every other notebook:

| Function                                               | Purpose                                                                 | Call timing                            |
|--------------------------------------------------------|-------------------------------------------------------------------------|----------------------------------------|
| log_start(run_id, pipeline_name, layer, source)        | Inserts a new row with status = 'running', start_time = now             | First line of main block               |
| log_end_success(run_id, pipeline_name, rows_processed) | Updates the row with status = 'success', end_time = now, rows_processed | Last line of main block (success path) |
| log_end_failure(run_id, pipeline_name, error_message)  | Updates the row with status = 'failed', end_time = now, error_message   | Inside exception handler               |

**Notebook integration pattern:**

```
run_id = <injected by pipeline parameter>
pipeline_name = 'spi_nb_bronze_energy'

log_start(run_id, pipeline_name, 'bronze', 'energy')
try:
    rows = main()
    log_end_success(run_id, pipeline_name, rows)
except Exception as e:
    log_end_failure(run_id, pipeline_name, str(e))
    raise
```

**9.3 Pipeline Integration**

Data Pipelines write their own log rows at the pipeline level (master start, layer start, layer end) via a small 'Stored procedure' or 'Script' activity. The `run_id` generated by the master pipeline is passed as a parameter to all child pipelines and notebooks.

**9.4 Querying the Log**

Typical diagnostic queries:

```
-- Last run summary
SELECT pipeline_name, layer, source, status,
       start_time, end_time, rows_processed
FROM spi_log_pipeline_execution
WHERE run_id = (SELECT MAX(run_id) FROM spi_log_pipeline_execution)
ORDER BY start_time;

-- Recent failures across all runs
SELECT pipeline_name, source, start_time, error_message
FROM spi_log_pipeline_execution
WHERE status = 'failed'
ORDER BY start_time DESC
LIMIT 20;
```

## 10. Power BI Technical Specification

The Power BI deliverable is a PBIP-format semantic model and report, connected to `spi_warehouse` via Direct Lake. This section specifies the technical configuration. The report's functional design (pages, visuals) is defined in Phase 3 Section 7 and is not repeated here.

### 10.1 Connectivity

**Direct Lake** is the chosen connectivity mode. Direct Lake loads data from the Warehouse's underlying Delta storage into VertiPaq on demand, without import refresh and without DirectQuery latency. It is the modern default for Fabric-native Power BI.

**Requirements for Direct Lake.** The Warehouse tables must use Delta storage (default in Fabric Warehouse), must not exceed workspace capacity limits, and must be in the same workspace as the semantic model (or in a workspace with the required cross-workspace configuration). All these conditions are met by the DEV and PROD workspace setup.

### 10.2 Semantic Model — Tables

The semantic model imports six tables from `spi_warehouse`: one fact and five dimensions.

| Model table     | Source table                                       |
|-----------------|----------------------------------------------------|
| Fact Indicators | <code>spi_warehouse.dbo.spi_fact_indicators</code> |
| Indicator       | <code>spi_warehouse.dbo.spi_dim_indicator</code>   |
| Region          | <code>spi_warehouse.dbo.spi_dim_region</code>      |
| Calendar        | <code>spi_warehouse.dbo.spi_dim_calendar</code>    |
| Measure Type    | <code>spi_warehouse.dbo.spi_dim_measure</code>     |
| Source          | <code>spi_warehouse.dbo.spi_dim_source</code>      |

Display names in the model use friendly casing (title case with spaces). Physical column names in the database remain snake\_case.

### 10.3 Relationships

| From (many)     | Column                     | To (one)     | Column                     | Cardinality | Cross-filter                    |
|-----------------|----------------------------|--------------|----------------------------|-------------|---------------------------------|
| Fact Indicators | <code>indicator_key</code> | Indicator    | <code>indicator_key</code> | Many-to-one | Single (Indicator filters Fact) |
| Fact Indicators | <code>region_key</code>    | Region       | <code>region_key</code>    | Many-to-one | Single                          |
| Fact Indicators | <code>calendar_key</code>  | Calendar     | <code>calendar_key</code>  | Many-to-one | Single                          |
| Fact Indicators | <code>measure_key</code>   | Measure Type | <code>measure_key</code>   | Many-to-one | Single                          |
| Fact Indicators | <code>source_key</code>    | Source       | <code>source_key</code>    | Many-to-one | Single                          |

All relationships are **single-direction**. Bi-directional filters are not used — they add complexity without benefit for this model and can cause ambiguity in DAX evaluation.

### 10.4 Calendar Dimension Marking

The `calendar` table is marked as the date table, even though it has monthly (not daily) granularity. `calendar_key` (YYYYMM integer) is used as the mark; time intelligence DAX functions work correctly at monthly grain when the key is properly ordered.

## 10.5 DAX Measures

The semantic model defines four primary measures that power most visuals. These are stable and can be authored as specified.

### 10.5.1 Indicator Value

```
Indicator Value =
SUM ( 'Fact Indicators'[indicator_value] )
```

Returns the value filtered by the current context. Used in all cards, charts, and tables.

### 10.5.2 Latest Period

```
Latest Period =
CALCULATE (
    MAX ( 'Calendar'[year_month] ),
    'Fact Indicators'[indicator_value] <> BLANK ()
)
```

Returns the most recent year\_month for which data exists, respecting current filters.

### 10.5.3 Latest Value

```
Latest Value =
CALCULATE (
    [Indicator Value],
    FILTER (
        ALL ( 'Calendar' ),
        'Calendar'[year_month] = [Latest Period]
    )
)
```

Returns the indicator value for the latest available period. Used in Overview KPI cards.

### 10.5.4 Trend Direction

```
Trend Direction =
VAR CurrentValue = [Latest Value]
VAR PreviousValue =
    CALCULATE (
        [Indicator Value],
        FILTER (
            ALL ( 'Calendar' ),
            'Calendar'[calendar_key] = MAX ( 'Calendar'[calendar_key] ) - 1
        )
    )
RETURN
    SWITCH (
        TRUE (),
        CurrentValue > PreviousValue, "Up",
        CurrentValue < PreviousValue, "Down",
        "Flat"
    )
```

Returns a string indicating trend direction vs. previous period. Used for trend arrows in KPI cards.

*Note on derived measures (YoY, MoM, YTD, QTD) — these are **not** computed as DAX measures; they are precomputed rows in the fact table (Phase 3 Section 5). The Power BI user selects the desired measure type via a slicer on the Measure Type dimension. DAX then simply filters Fact Indicators by measure\_key and aggregates indicator\_value.*

## 10.6 Report File Format — PBIP

The report is saved in **PBIP (Power BI Project)** format rather than PBIX. PBIP decomposes the semantic model into TMDL files and the report into JSON, enabling Git version control with meaningful diffs. The PBIP folder structure is:

```
/src/power-bi/  
■■■ spi_spain_indicators.Report/  
■■■ spi_spain_indicators.SemanticModel/  
■■■ .gitignore  
■■■ spi_spain_indicators.pbip
```

## 10.7 Report Pages

Six pages per Phase 3 Section 7: Overview, Consumer Prices, Industrial Production, Energy Demand, Construction Tenders, Tax Revenue. The technical implementation details (visual configurations, slicers, bookmarks) are finalized in Phase 5 and do not warrant pre-specification here.

## 11. Deployment Strategy

The project is developed in DEV and promoted to PROD near the end of Phase 5. This section specifies the promotion mechanism.

### 11.1 Fabric Deployment Pipelines

Fabric Deployment Pipelines provide a native mechanism to promote artifacts between workspaces. The pipeline binds the DEV workspace as Stage 1 and the PROD workspace as Stage 2.

| Stage       | Workspace                 |
|-------------|---------------------------|
| Development | spi-spain-indicators-dev  |
| Production  | spi-spain-indicators-prod |

An intermediate Test stage is not used — the portfolio scope does not justify a three-stage promotion flow.

### 11.2 What Gets Promoted

| Artifact type                    | Promoted?                          |
|----------------------------------|------------------------------------|
| Lakehouse (empty shell)          | Yes                                |
| Warehouse (empty shell with DDL) | Yes                                |
| Notebooks                        | Yes                                |
| Pipelines                        | Yes                                |
| Dataflows Gen2                   | Yes                                |
| Semantic Model                   | Yes                                |
| Power BI Report                  | Yes                                |
| Data (Bronze, Silver, Gold rows) | No — PROD regenerates its own data |

Deployment Pipelines promote artifact definitions only. PROD starts empty and is populated by executing the master pipeline with `data_scope = full_history`.

### 11.3 Deployment Rules (Parameter Substitution)

Deployment rules substitute environment-specific values during promotion. The connection to Azure SQL uses a different credential in PROD than in DEV (optional but recommended); the pipeline parameters also differ.

| Parameter            | DEV value                     | PROD value                     |
|----------------------|-------------------------------|--------------------------------|
| environment          | dev                           | prod                           |
| data_scope           | last_24_periods               | full_history                   |
| Azure SQL connection | conn_azure_sql_ipi_source_dev | conn_azure_sql_ipi_source_prod |

### 11.4 Deployment Sequence

1. DEV workspace is stable; all pipelines run end-to-end successfully.
2. PROD workspace is created and bound to the Deployment Pipeline as Stage 2.
3. Deployment rules are configured in Fabric's Deployment Pipelines UI.

4. 'Deploy' is triggered from DEV → PROD. All artifacts are copied.
5. The master pipeline in PROD is triggered manually with `data_scope = full_history`.
6. Screenshots are captured in PROD for the portfolio. DEV remains as the active editing surface.



## 12. Testing and Validation Plan

Validation is performed per layer using a combination of row-count reconciliation, sample inspection, and data quality rule verification.

### 12.1 Bronze Validation

| Check                                        | Query pattern                                           |
|----------------------------------------------|---------------------------------------------------------|
| Row count per source matches expected volume | SELECT COUNT(*) FROM                                    |
| No rows missing the ingestion timestamp      | SELECT COUNT(*) FROM WHERE _ingestion_timestamp IS NULL |
| Source file / table identifier populated     | SELECT DISTINCT _source_file FROM                       |
| Sample rows match source file content        | Visual comparison of 10 rows vs. raw file               |

### 12.2 Silver Validation

| Check                                                            | Query pattern                                                                                                                |
|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Row count reduced from Bronze in an expected way (after filters) | Compare COUNT(*) before/after; justify the difference                                                                        |
| Geographic filter applied: only three regions                    | SELECT DISTINCT region_name FROM must return exactly the three canonical values                                              |
| Year and month parsed correctly                                  | SELECT MIN(year), MAX(year), MIN(month), MAX(month) FROM                                                                     |
| No null indicator_value where business rules require non-null    | Check DQ-03 per source                                                                                                       |
| No duplicates on natural key                                     | SELECT region_name, year, month, indicator_category, COUNT(*) FROM GROUP BY 1,2,3,4 HAVING COUNT(*) > 1 — should return zero |

### 12.3 Gold Validation

| Check                                                  | Query pattern                                                         |
|--------------------------------------------------------|-----------------------------------------------------------------------|
| Every fact row has valid FKs                           | Left join to each dimension; count nulls                              |
| Composite PK is unique                                 | GROUP BY all five keys, HAVING COUNT(*) > 1 returns zero              |
| Five measure types present per indicator-region-period | For a sample combination, SELECT DISTINCT measure_key should return 5 |
| Derived measure nulls conform to business rules        | Inspect NULL patterns per Phase 3 Section 5                           |
| Row count roughly equals (Silver IDX rows) × 5         | Rough sanity check of derived measure expansion                       |

### 12.4 End-to-End Validation

| Check                                           | Approach                                                              |
|-------------------------------------------------|-----------------------------------------------------------------------|
| Power BI cards show correct values              | Compare Power BI card value to equivalent SQL query against Warehouse |
| Slicer changes propagate correctly              | Change region slicer; verify card updates                             |
| Measure Type slicer switches measures correctly | Toggle IDX → YoY → MoM; verify chart values change appropriately      |

## 12.5 Pre-PROD Checklist

Before deploying DEV to PROD, the following checklist must pass:

- All notebooks complete successfully with `full_history` scope.
- All pipelines in the master flow complete with success status.
- Log table has one row per layer-source combination per run, with no orphaned running statuses.
- Gold fact row count matches expected volume (approximately 300,000 rows).
- All Power BI pages render without errors.
- DAX measures return non-blank values for realistic slicer selections.

## 13. Troubleshooting Guide

This section anticipates errors that are likely to appear during Phase 5, based on the characteristics of each source and the Fabric components involved. Additional troubleshooting entries are added during Phase 5 as real errors are encountered.

***Living section** — this is the starting set. Phase 5 will expand it with hands-on findings and solutions. The expanded version is versioned in the [GitHub repository](#).*

### 13.1 Encoding — ISO-8859-15 CSV Files

**Symptom.** Spanish characters (ñ, á, é, ü) appear garbled in Bronze or Silver tables. Example: 'Cataluña' becomes 'Cataluña'.

**Cause.** The INE CSV is encoded in ISO-8859-15; the ingestion code is reading it as UTF-8.

**Resolution.** In the Copy Activity or notebook, set `encoding = 'iso-8859-15'` explicitly at read time. Write to Bronze as UTF-8. Verify by querying `SELECT DISTINCT region_name` in Silver.

### 13.2 Merged Cells — XLS Legacy

**Symptom.** The construction XLS files parse into DataFrames with NaN values in columns where merged cells should propagate.

**Cause.** `xlrd` does not automatically fill merged cells; the value lives in the top-left cell of the merge range only.

**Resolution.** Detect merged ranges via `sheet.merged_cells` and forward-fill the value across the range before parsing the DataFrame.

### 13.3 API Timeout — REE Energy

**Symptom.** HTTP 504 or connection reset when fetching a full year of monthly data.

**Cause.** Large date ranges in a single request can exceed the API's processing window.

**Resolution.** The notebook retry logic (Section 5.2) handles this automatically for 3 attempts with exponential backoff. If all retries fail, split the request into smaller ranges (e.g., 6-month windows) as a fallback.

### 13.4 Direct Lake — Fallback to DirectQuery

**Symptom.** Power BI visuals show slower response than expected; the semantic model indicates 'DirectQuery fallback' has occurred.

**Cause.** Direct Lake has row-count and data-type limits per visual query. When a query exceeds those limits, the engine falls back to DirectQuery against the SQL endpoint, which is slower.

**Resolution.** Check the 'Direct Lake' status in the semantic model settings. Reduce the cardinality of slicers, or simplify overly complex DAX measures. For the portfolio, occasional fallback is acceptable and does not need to be eliminated.

### 13.5 Cross-Tenant Auth Failure

**Symptom.** The Fabric Copy Activity against Azure SQL fails with 'Login failed for user' or a token-related error.

**Cause.** Using Entra ID authentication on a cross-tenant connection. Fabric's token is tenant-scoped and is not valid against a personal-tenant SQL server.

**Resolution.** Switch the connection to SQL Authentication per Section 8. Confirm the `svc_fabric_reader` user exists and has SELECT permission on `stg_ipi_raw`.

### 13.6 Dataflow Gen2 — Destination Not Writing

**Symptom.** A Dataflow Gen2 runs successfully but the destination table appears empty or outdated.

**Cause.** The destination step is configured with 'Append' semantics but the preview was tested against a snapshot; or the destination settings were not saved before publishing.

**Resolution.** Verify destination configuration uses 'Replace' (truncate and reload) for Silver and Gold layers. Re-publish the dataflow. Execute from the pipeline, not the preview pane.

### 13.7 Trial Capacity Exhaustion

**Symptom.** Pipelines queue indefinitely or fail with 'capacity limit reached' errors.

**Cause.** The Fabric trial capacity has a fair-use limit; concurrent heavy operations can temporarily exhaust it.

**Resolution.** Wait 15-30 minutes and retry. Reduce concurrency by running pipelines sequentially instead of parallel. If persistent, contact workspace admin to verify the trial is still active.

## 14. Next Steps

With the technical specification formalized, the project proceeds to **Phase 5: Development in Fabric**, which will deliver:

- Activation of the Fabric Trial Capacity and creation of the DEV workspace.
- Implementation of all Bronze, Silver, and Gold components per the specifications in this document.
- Power BI semantic model and report in PBIP format.
- Export of all artifacts from Fabric to the local repository structure.
- Capture of screenshots evidencing the working solution in PROD.

### Phase Progress

|   |                                                                                                 |          |
|---|-------------------------------------------------------------------------------------------------|----------|
| 0 | <b>Context &amp; Strategy</b><br>Deliverable: Project Brief (PDF)                               | COMPLETE |
| 1 | <b>Requirements &amp; Source Catalog</b><br>Deliverable: Requirements Document + Source Catalog | COMPLETE |
| 2 | <b>Architecture Design</b><br>Deliverable: Architecture Design Document                         | COMPLETE |
| 3 | <b>Functional Specification</b><br>Deliverable: Functional Specification (PDF)                  | COMPLETE |
| 4 | <b>Technical Specification</b><br>Deliverable: Technical Specification (PDF)                    | COMPLETE |
| 5 | <b>Development in Fabric</b><br>Deliverable: Working Solution + Code                            | NEXT     |
| 6 | <b>GitHub Publication</b><br>Deliverable: GitHub Repository + README                            | PENDING  |

Document version 1.0 | April 2026 | Phase 4 deliverable